

Fortgeschrittene Informatik

Prof. Dr. Andreas Schilling

www.ravensburg.dhbw.de

Thymeleaf

Grundlagen

Grundlagen zur
Erstellung dynamischer
Websites.

Statische Website

Nutzung von Thymeleaf zur
Anzeige statischer HTML-
Dateien.

Dynamische Website

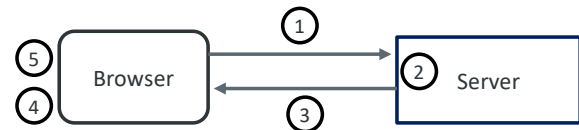
Nutzung von Thymeleaf
zur Anzeige dynamischer
Laufzeitinformationen.

Grundlagen

Übersicht

Der **Weg zur Darstellung** einer interaktiven Website

1. Der Client sendet eine **HTTP-Anfrage** zum Server.
2. Der **Server bearbeitet** die Anfrage.
3. Die **HTTP-Antwort** wird übertragen.
4. Der **Browser verarbeitet** die Antwort.
5. Der Browser lädt **weitere Bibliotheken** (CSS/JS).
6. Die Website kann **interaktiv** genutzt werden



Welcher dieser Verarbeitungsschritte kann optimiert werden?

Der generelle Ablauf zur Darstellung einer interaktiven Website besteht aus 6 grundlegenden Schritten:

1. Die Übertragung der HTTP-Anfrage. Nachdem der Nutzer die Internetadresse der aufzurufenden Website in seinem Browser bestätigt, sendet der Browser eine HTTP-GET Abfrage zur eingegebenen Adresse.
2. Im Anschluss an die Übermittlung, bearbeitet der Server die Anfrage (bspw. Pfad-Auflösung, Durchführung von Berechnungen, Konvertierung der Ergebnisse in das Antwortformat, etc.)
3. Die erstellte Serverantwort wird zum Aufrufer übertragen.
4. Der Browser wertet die Serverantwort aus und lädt ggf. verlinkte Ressourcen wie CSS- und JavaScript-Dateien.
5. Nach Erhalt der benötigten Dateien stellt der Browser die interaktive Website dar.

Die sofortige Darstellung von interaktiven Websites ist eine Grunderwartung vieler Nutzer. Webentwickler stellen sich daher seit Jahrzehnten die Frage, wie sich die oben skizzierten Verarbeitungsschritte optimieren lassen um so die benötigte Darstellungszeit einer Website zu minimieren.

Grundlagen

Static Server-Side Rendering

Ablauf des Static Server-Side Rendering

1. Der Client sendet eine HTTP-Anfrage zum Server.
2. Der Server beantwortet die Anfrage **ausschließlich mit statischen Seiten** (reduzierte Last).
3. HTTP-Antwort wird übertragen.
4. Der Browser verarbeitet Antwort.
5. Browser lädt weitere Bibliotheken (CSS/JS).
6. Website kann interaktiv genutzt werden



→ Design digitaler Visitenkarten



- + SEO-optimized
- + Caching der Antwort
- + Schnelle Anzeige
- Keine Interaktivität

Bei der Herangehensweise des Static Server-Side Renderings (Static-SSR) werden vom Server ausschließlich statische HTML-Ressourcen zurückgeliefert. Dadurch entfällt die individuelle Berechnung auf dem Server. Stattdessen übt der Server nur die Funktion eines Webserver aus, der bereitgestellte Dateien zurückgibt wodurch sich die Bearbeitungszeit massiv reduziert.

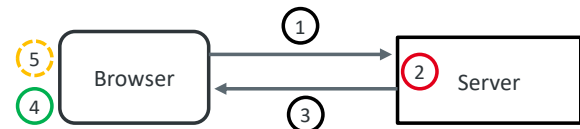
Neben der schnellen Ladezeit der HTML-Seite ermöglicht dieser Ansatz, dass der Webseiteninhalt leicht von Suchmaschinen erfasst und verarbeitet werden kann (Search Engine Optimization – SEO). Dies ist die Grundlage dafür, dass die Website für relevante Suchanfragen als Antwort aufgelistet wird. Static Server Side Rendering hat zudem den Vorteil, dass die Antwort des Servers zwischengespeichert („gecacht“) werden kann, wodurch sich die Ladezeit weiter verkürzt. Jedoch muss festgehalten werden, dass sich Static Server-Side Rendering nur für wenige Einsatzgebiete eignet, da keine Interaktivität unterstützt wird. Stattdessen wird jedem Nutzer die gleiche HTML-Antwort gegeben. Typische Einsatzgebiete von Static Server-Side Rendering sind Websites die als digitale Visitenkarten für Personen und Unternehmen dienen.

Grundlagen

Server-Side Rendering

Ablauf des Server-Side Rendering

1. Der Client sendet eine HTTP-Anfrage zum Server.
2. Der Server liefert die fertig darzustellende Seite zurück (**erhöhte Arbeitslast**).
3. HTTP-Antwort wird übertragen
4. Der Browser verarbeitet Antwort (**reduzierte Last**)
5. Browser lädt ggf. weitere Bibliotheken (CSS/JS).
6. Website kann interaktiv genutzt werden



- + SEO-optimized
- + Schnelle Anzeige
- Server als Bottleneck
- Keine echte Interaktivität

→ Design traditioneller Webshops



Beim Server-Side Rendering wird eine interaktive Website durch Berechnungen auf dem Server umgesetzt. Hierzu wird im zweiten Schritt die Anfrage vom Server mit Hilfe von Datenbankabfragen bearbeitet und die anzuzeigende Website zusammengebaut. Dadurch, dass der Browser eine fertige HTML-Seite serviert bekommt, reduziert sich sein Aufwand zur Darstellung und es müssen nur noch ggf. weitere Bibliotheken geladen werden.

Das Server-Side Rendering ist für die Optimierung von Suchmaschinenanfragen prädestiniert. Dadurch, dass der Server auf eine Anfrage die fertige HTML-Seite zurückgibt, die von Suchmaschinen ausgewertet werden kann, wird die Website bei relevanten Suchanfragen als Antwort aufgelistet. Des Weiteren sticht die Herangehensweise durch eine schnelle Anzeige im Browser hervor, da im Browser kaum Berechnungen vorgenommen werden. Der Nachteil dieser Herangehensweise ist, dass der Server zum Engpass bei zeitgleichen Anfragen wird. Ferner kann keine „echte“ Interaktivität umgesetzt werden, da für jeden Seitenaufruf auf die Serverantwort gewartet werden muss. Typische Einsatzgebiete des Server-Side Renderings sind Webshops.

Grundlagen

Client-Side Rendering

Der Weg zur Darstellung einer interaktiven Website

1. Der Client sendet eine HTTP-Anfrage zum Server.
2. Server als Dateiserver für JS-Dateien (**reduzierte Last**).
3. HTTP-Antwort wird übertragen.
4. Der Browser führt JS-Dateien aus (**erhöhte Last**).
5. Browser führt ggf. REST-Aufrufe aus um die Elemente darzustellen (**erhöhte Last**).
6. Website kann interaktiv genutzt werden



- + Reibungslose Interaktivität
- + Entlastung des Servers
- Langsame Anzeige
- Kein SEO

→ Design von App-Websites



Die dritte Herangehensweise zur Darstellung interaktiver Websites ist das Client-Side Rendering. Bei dieser Darstellungsart wird der Server grundlegend entlastet. Statt der fertig darzustellenden Website liefert der Server eine umfassende JavaScript-Datei zurück. Diese JavaScript-Datei wird im Browser ausgeführt um die darzustellende Seite zu bauen. Hierzu führt der Browser ggf. REST-Aufrufe durch um die notwendigen Daten zur Verarbeitung zu erhalten. In Folge dessen steigt die Last im Browser merklich und die Erstladezeit der Website hängt maßgeblich von der Internetverbindung des Nutzers ab.

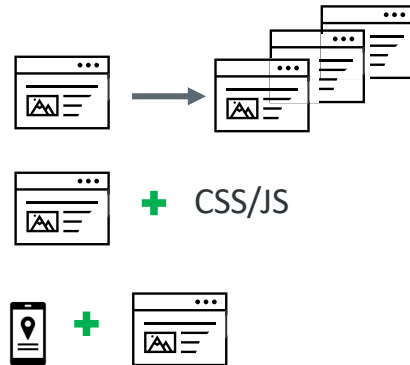
Der Vorteil des Client-Side Renderings ist die reibungslose Interaktivität nach dem ersten Laden der Website, da ggf. auf weitere Serverzugriffe verzichtet werden kann. Ferner wird der Server massiv entlastet, da er nur noch die Aufgabe eines Dateiservers erfüllt. Der Nachteil des Client-Side Renderings ist eine langsame Erstanzeige der Website und der Wegfall der Unterstützung von Suchmaschinenabfragen. Suchmaschinen werten in der Regel kein JavaScript aus, sodass sie die Relevanz der Website für Suchanfragen nicht bewerten können. Typische Einsatzgebiete von Client-Side Rendering sind Websites, die eine Interaktivität ähnlich zu Apps besitzen (bspw. Instagram).

Grundlagen

SSR-CSR Kombinationen

Aufgrund der individuellen Vor- und Nachteile werden SSR und CSR heute für viele Anwendungsfälle kombiniert.

- Web-Frameworks wandeln **dynamischen Inhalt in statische Websites** um.
- Dynamische Websites mit **SSR binden weitere Bibliotheken ein** um die Interaktivität zu verbessern.
- CSR-Websites nutzen ein **Grundgerüst auf dem Server** um SEO zu ermöglichen.
- **Gefahr:** 1 Website zum Preis von 2



Da Client-Side und Server-Side Rendering individuelle Vor- und Nachteile haben, werden heute oft Kombinationen der beiden Extreme eingesetzt. Eine Entwicklung ist beispielsweise, dass dynamische Websites, die eine geringe Änderungsfrequenz aufweisen (bspw. Blogs), mithilfe von Frameworks zu statischen Websites umgewandelt werden. Dadurch reduziert sich maßgeblich die Last auf dem Server zur Bearbeitung der Anfrage.

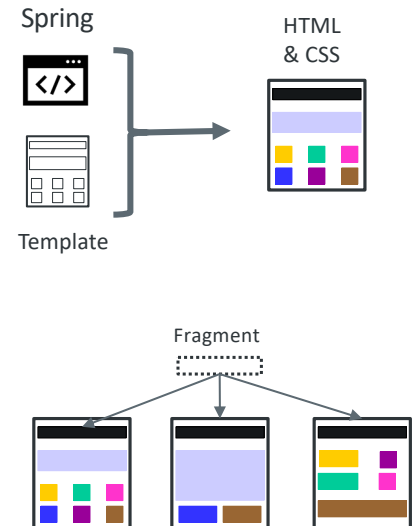
Eine weitere Entwicklung ist, dass beim Server-Side Rendering nicht die gesamte Funktionalität der Website mit jedem Aufruf auf dem Server gebaut wird. Statt dessen werden CSS- und JS-Dateien verlinkt, die zur Darstellung und Interaktivität der Website verwendet werden. Hierdurch wird ein Teil der Last vom Server in den Browser verlagert und die Interaktivität wird verbessert.

Abschließend gibt es im Falle von Client-Side Rendering die Möglichkeit ein generisches Grundgerüst der zurückgegebenen Antwort auf dem Server zu platzieren, die von Suchmaschinen ausgewertet werden kann. So kann auch im Falle des Client-Side Rendering eine Optimierung der Suchmaschinenabfragen umgesetzt werden.

Thymeleaf

Grundlagen

- **Thymeleaf** ist ein Java-Framework welches SSR nutzt um dynamische Websites zu erstellen.
- Um die Bearbeitungszeit auf dem Server so gering wie möglich zu halten verwendet Thymeleaf **HTML-Vorlagen**, die zur Laufzeit „befüllt“ werden.
- Diese HTML-Vorlagen (auch „natural templates“) bilden das **Bindeglied** zwischen **Web-Designern** und **Programmierern**.
- Ein Eckpfeiler von Thymeleaf ist die Untergliederung einer Website in **kleine, wiederverwendbare Codeblöcke**, den sogenannten **Fragmenten**.

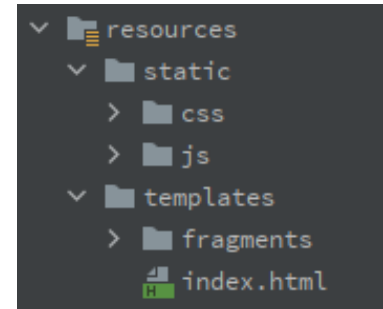


Wir verwenden im Folgenden zur Umsetzung einer dynamischen Website das Java-Framework Thymeleaf, das auf dem Prinzip des Server-Side Rendering basiert. Zur Minimierung der Serverlast setzt Thymeleaf HTML-Vorlagen ein, die zur Laufzeit mit dynamischen Daten befüllt werden. Diese HTML-Vorlagen werden bei Thymeleaf „natural templates“ genannt, da sie auch ohne Java-Laufzeitumgebung im Browser dargestellt werden können. Dieser Mechanismus ist zentral für die Zusammenarbeit von Web-Designern und Backend-Programmierern, da so deren Abstimmung maßgeblich vereinfacht wird. Ein weiterer zentrale Baustein von Thymeleaf sind Fragmente. Hierunter versteht man kleine wiederverwendbare Bausteine, die für die Umsetzung der Websites verwendet werden. Dieser Mechanismus ermöglicht eine hohe Flexibilität, da es damit nicht mehr zwingend notwendig ist, dass der HTML-Code jeder Seite komplett ausgeschrieben werden muss.

Thymeleaf

Ordnerstruktur

- Thymeleaf wird in ein Spring-Projekt über den „**resources**“ Ordner integriert.
- Der Ordner „**static**“ beinhaltet all jene Ressourcen die von Thymeleaf nicht angepasst werden, aber für die Darstellung der Websites benötigt werden (bspw. CSS, JS, Bilder).
- Der Ordner „**templates**“ beinhaltet alle von Thymeleaf verwendeten Vorlagen und Bausteine.
 - Im **Wurzelverzeichnis** befinden sich die Vorlagen der Websites die zur Laufzeit bereitgestellt werden.
 - Im Verzeichnis **fragments** befinden sich die Bausteine, die in die einzelnen Websites eingebettet werden können.



Für die Verwendung von Thymeleaf ist es erforderlich zwei neue Unterordner im Ressourcenverzeichnis der Spring-Anwendung anzulegen. Im Ordner static werden jene Ressourcen abgelegt, die zur Laufzeit nicht verändert werden. Hierunter fallen beispielsweise die CSS- und JS-Dateien aber auch Bilder. Zur Übersichtlichkeit und aus Best-Practice Erfahrungen wird der static-Ordner je nach Dateityp in weitere Unterordner untergliedert.

Im Ordner templates werden die HTML-Vorlagen gesammelt, die zur Laufzeit mit Java-Objekten angereichert werden. Hierzu werden zur Laufzeit alle in diesem Verzeichnis verfügbaren HTML-Dateien als Websites zugreifbar gemacht. Im Ordner fragments werden die wiederverwendbaren Bausteine gesammelt, die auf den einzelnen HTML-Seiten eingebettet werden.

Grundlagen

Grundlagen zur Erstellung dynamischer Websites.

Statische Website

Nutzung von Thymeleaf zur Anzeige statischer HTML-Dateien.

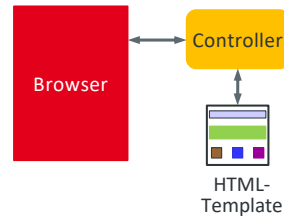
Dynamische Website

Nutzung von Thymeleaf zur Anzeige dynamischer Laufzeitinformationen.

Statische Websites

Integration in Spring

- Thymeleaf integriert sich nahtlos in die **Schichtenarchitektur** von Spring.
- Zur Rückgabe der HTML-Websites ist ein **eigener Controller** notwendig, der **eine Methode je anzusprechender Website** besitzt.
- Über die Annotation **GetMapping** wird der HTTP-Zugriffspfad der Methode spezifiziert.
- Der **Rückgabestring** der Methoden, wird von Spring benutzt um das **zu verwendende HTML-Template** zu identifizieren.



```

@Controller
public class ThMovieControllerImpl implements ThMovieController {

    @GetMapping(value={"/contact"})
    public String contactInfo() { return "contact"; }
  }
  
```

Ausschnitt der Klasse ThMovieControllerImpl

Gemäß der Schichtenarchitektur von Spring, ist es für die Nutzung der HTML-Templates erforderlich eine neue Controllerklasse anzulegen, da hier eine direkte Interaktion mit dem Nutzer geschieht. Im Rahmen des neu angelegten Controllers, wird für jede anzusprechende HTML-Seite eine eigene Methode verfasst. Die GetMapping Annotation spezifiziert dabei die Adresse unter der die HTML-Seite erreicht wird. Handelt es sich bei der referenzierten HTML-Seite um eine Website mit statischen Inhalten (bspw. das Impressum) muss die Methode den Dateinamen des Templates als String zurückgeben. Spring kümmert sich anschließend um die Auswahl und Rückgabe des entsprechenden HTML-Template zur Laufzeit. Hierbei ist zu beachten, dass der Dateiname nicht vollständig sondern nur eindeutig spezifiziert werden muss.

Statische Websites

Natural Template

- **Import des th-Namespaces** damit Spring zur Laufzeit den HTML-Code abändern kann. Beim standalone Zugriff werden alle th-Elemente ignoriert.
- Die **th-Elemente** ermöglichen es HTML-Elemente zu **überschreiben** und neue Elemente **einzufügen**.
- **Fragmente** werden mit dem Befehl `th:insert` gefolgt vom Dateipfad und dem Bausteinnamen eingefügt.
- Der **Zugriffspfad auf die Fragmente** wird vom **Template-Ordner** aus verfasst.

```
<!doctype html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="utf-8">
  <title>Contact Details</title>
  <!-- Bootstrap core CSS -->
  <link th:href="@{css/bootstrap.min.css}" href="https://cdn.jsdelivr.net
    rel="stylesheet">
</head>
<body>

<main role="main">
  <section th:insert="fragments/banner :: banner"></section>
  <section>My contact</section>
  <p>Max Mustermann <br> Musterstraße 2, <br> 32421 Musterhausen</p>
</main>

<footer th:insert="fragments/footer :: footer"></footer>

<!-- Bootstrap core JavaScript -->
<script th:src="@{js/bootstrap.bundle.min.js}" src="https://cdn.jsdelivr.n
</body>
</html>
```

Ausschnitt der Klasse `contact.html`

13

Der Grundgedanke der „natural templates“ ist, dass die HTML-Vorlagen sowohl mit als auch ohne Spring Laufzeit angezeigt werden können. Hierzu wird folgender Trick verwendet:

Im obersten HTML-Tag wird der `th`-Namespace importiert der die Elemente zur dynamischen Abänderung der HTML-Seite umfasst. Diese `th`-Elemente werden jedoch nur von Spring ausgewertet. Beim standalone Zugriff werden die `th`-Elemente ignoriert, da der Browser sie nicht auswerten kann. So ist es beim vorliegenden Codeauszug möglich, die remote gehosteten Bootstrap Bibliotheken für den standalone Zugriff und die lokal gehosteten Bibliotheken zur Laufzeit von Spring zu verwenden.

Fragmente werden mit der Befehl `th:insert` in den HTML-Code eingefügt. Hierbei wird der Dateipfad und der konkrete Bausteinname referenziert. Zur Laufzeit von Spring wird dieser Baustein in die HTML-Vorlage eingefügt. Der Dateipfad zu den Fragmenten ist ausgehend vom Templates Ordner zu spezifizieren.

Statische Website

Hands-On

- Ordnerstruktur
- Aufbau und Funktionsweise des Controllers
- Natural-Templates
 - Standalone Design
 - th-Replace Syntax
 - Nutzung von Fragmenten
- Lokale vs. CDN-Einbindung der Bootstrap Bibliothek

*Standalone-
Ansicht*

My contact
Max Mustermann
Musterstraße 2,
32421 Musterhausen



*Server-Side
Rendering*

My Movie Database

This is a simple demo app to catalogue a movie collection.

My contact
Max Mustermann
Musterstraße 2,
32421 Musterhausen

[Back to top](#)

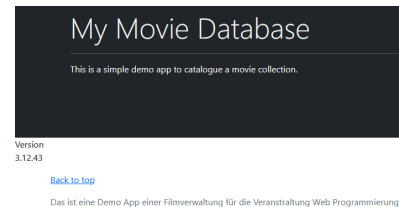
Das ist eine Demo App einer Filmverwaltung für die Veranstaltung Web Programmierung

Aufgabe 10

- a) Erstellen Sie bitte die neue statische Seite „version.html“ die ähnlich zur „contacts.html“ aufgebaut ist und die statische Versionsnummer „3.12.43“ anzeigt.

Die neue Seite soll unter der Adresse:
<http://localhost:8080/version> zurückgegeben werden.

- b) Ändern Sie bitte das Template „contact.html“ so ab, dass das Banner sowohl in der standalone Ansicht als auch zur Spring Laufzeit angezeigt wird.



Grundlagen

Grundlagen zur Erstellung dynamischer Websites.

Statische Website

Nutzung von Thymeleaf zur Anzeige statischer HTML-Dateien.

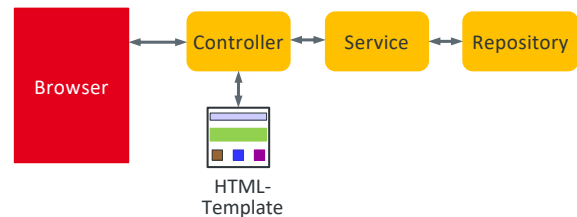
Dynamische Website

Nutzung von Thymeleaf zur Anzeige dynamischer Laufzeitinformationen.

Dynamische Websites

Integration in Spring

- Zur Anzeige von **Laufzeitinformationen** ruft der Controller die **Funktionen des Service** auf und bettet dessen Antwort in das HTML-Template ein.
- Die **Rückgabe** der Controllermethode ist ein **ModelAndView** Objekt.
- Das **ModelAndView** Objekt wird mit dem **Namen des HTML-Template** instanziiert.
- Laufzeitobjekte werden dem **ModelAndView** Objekt mit der Methode **addObject** **benannt übergeben**.



```
@Override
@GetMapping(value = {"", "/"}, produces = {"text/html"})
public ModelAndView index() {
    List<MovieDTO> movies = this.movieService.getAllMovies();

    ModelAndView modelAndView = new ModelAndView("index");
    modelAndView.addObject("data", movies);
    return modelAndView;
}
```

Ausschnitt der Klasse ThMovieControllerImpl

17

Zur Darstellung von Laufzeitinformationen ruft der Controller zunächst die korrespondierenden Servicemethoden auf und integriert die Rückgaben dieser in das zu verwendende HTML-Template. Wichtig hierbei ist, dass in diesem Fall die Controllermethode kein String mehr zurückgibt, sondern ein ModelAndView Objekt. Dieses Objekt wird im Controller mit dem Dateinamen der zu verwendenden HTML-Vorlage instanziiert. Der Dateiname muss dabei nicht vollständig sondern nur eindeutig spezifiziert werden. Zum Abschluss werden dem ModelAndView Objekt durch die Methode addObject, die zu verwendenden Laufzeitinformationen benannt übergeben.

Dynamische Websites

Darstellung von Objektinformationen

- Der Zugriff auf die Laufzeitinformationen im HTML-Template erfolgt im Format **`${Objektname.Attribut}`**
- Der **Objektname** muss **identisch zur Benennung** des Objekts im Controller sein.
- Durch **th:each** wird eine **Schleife** umgesetzt, die den gesamten Block, in dem die Anweisung steht, für jedes Element wiederholt.
- Durch **th:text** wird der dargestellte Text eines HTML-Elements überschrieben.
- Attributwerte können mit festen Strings im Format **`„${objekt.attribut + ,string}“`** kombiniert werden.

Ausschnitt der Klasse *ThMovieControllerImpl*

```
@Override
@GetMapping(value = {"/", "index.html"})
public ModelAndView index(){
    List<MovieDTO> movies = this.movieService.getAllMovies();

    ModelAndView modelAndView = new ModelAndView( "viewName: index");
    modelAndView.addObject( attributeName: "data" movies);
    return modelAndView;
}
```

```
<div class="col-md-4" th:each="item : ${data}">
  <div class="card mb-4 box-shadow">
    
      <p class="card-text" th:text="${item.title}">Interstellar</p>
      <p class="card-text" th:text="${IMDB: ' + item.imdbRating}">IMDB: 8
      <div class="d-flex justify-content-between align-items-center">
        <div class="btn-group">
          <button type="button" class="btn btn-sm btn-outline-secondary
            data-bs-target="#exampleModal" th:data-bs-target="${
          </button>
        </div>
        <small class="text-muted" th:text="${item.duration} + ' Stunden'
      </div>
    </div>
  </div>
</div>
<div th:insert="fragments/movie-modal :: movie-modal"></div>
```

Ausschnitt des Fragments *grid.html*

18

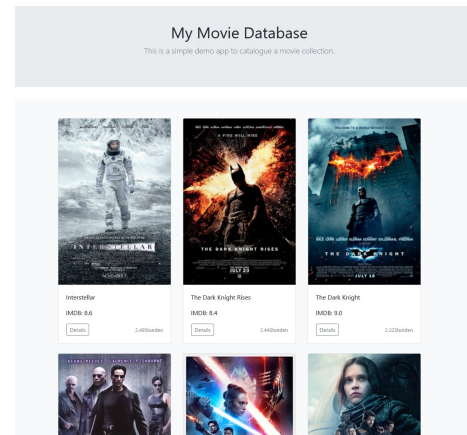
Schauen wir uns zum Abschluss an wie Java-Objekte, in die dargestellte HTML-Seite integriert werden können. Wichtig hierbei ist, dass der Objektname, der im Rahmen der `addObject` Methode im Controller gewählt wurde, auch im HTML-Template verwendet wird. Konkret kann auf das Objekt, oder einzelne Eigenschaften dessen, im Format **`„${Objektname.Attributname}“`** zugegriffen werden. Handelt es sich bei dem Objekt um eine Liste, kann mit dem Befehl **th:each** eine Schleife umgesetzt werden, die einen bestimmten Block im HTML-Code für jedes Listenelement wiederholt. Mit dem Befehl **th:text** kann der dargestellte Text des HTML-Elements überschrieben werden. Hierbei ist es auch möglich Attributwerte um statische Texte zu ergänzen (bspw. Währungskürzel). Dies erfolgt im Format **`„${objekt.attribut + ,string}“`**.

Dynamische Websites

Hands-On

Darstellung der Filmübersicht

- **Controller**
 - Serviceaufruf
 - Erstellung des ModelAndView Objekts
 - Objektübergabe
- **Template**
 - Auswahl aus [Bootstrap Beispielen](#)
 - Modifikation um Laufzeitinformationen



Aufgabe 11

Thymeleaf

- a) Erweitern Sie die Demo App zur Filmverwaltung, sodass im Modal zur Detailübersicht der Filme eine Tabelle mit den Auszeichnungen für den Film angezeigt wird. Bitte achten Sie darauf, eine **dynamische** und eine **statische Version** des angepassten Modals zu erstellen.



When the menace known as the Joker wreaks havoc and chaos on the people of Gotham, Batman must accept one of the greatest psychological and physical tests of his ability to fight injustice.

IMDB: 9.0

2.32 Stunden

Trailer

Close



When the menace known as the Joker wreaks havoc and chaos on the people of Gotham, Batman must accept one of the greatest psychological and physical tests of his ability to fight injustice.

IMDB: 9.0

2.32 Stunden

Academy	Category	Celebration Year
Oscar	Best Performance by an Actor in a Supporting Role	2009
Oscar	Best Achievement in Sound Editing	2009

Aufgabe 11

Thymeleaf

- b) Nutzen Sie ein Badge Button im Modal der Detailinformationen des Films mit dem Sie die Anzahl der Auszeichnungen angeben. Überführen Sie die Tabelle der Auszeichnungen in einen eigenen Modal, der sich öffnet sobald man auf dem Badge Button klickt



When the menace known as the Joker wreaks havoc and chaos on the people of Gotham, Batman must accept one of the greatest psychological and physical tests of his ability to fight injustice.

IMDB: 9.0

2.32 Stunden



Academy	Category	Celebration Year
Oscar	Best Performance by an Actor in a Supporting Role	2009
Oscar	Best Achievement in Sound Editing	2009

Close